MEMÓRIA · ENCONTRO 3 · LIGA PROTOFPGA

Gabarito

Parte A — Atividade em aula

Item	Resposta
1	O 9 está em **`0x10010008`** (linha 0x10010000, coluna Value (+8)). O 5 está em **`0x10010010`** (coluna Value (+10) — lembre que +10 é hexadecimal, 16 bytes).
2	Duas linhas: `lui $1,4097` e `ori $16,$1,0`. Com a exibição em hexadecimal ligada, aparecem como `lui $1,0x00001001` e `ori $16,$1,0x00000000`.
3	`$s0` = 268500992. Não é o 7: é `0x10010000` em decimal, o **endereço** onde o 7 está.
4	`$t2` = 26. Run I/O: `Soma = 26`. Muda a palavra em **`0x10010014`** (`soma`), de 0 para 26.
6	Erro mais comum: prever `$s0` = 7. Segundo mais comum: achar que o `sw` escreve em `$t2`, ou que ele altera alguma palavra do vetor. Nenhuma das cinco palavras do vetor muda.
7	Run I/O: `Soma = 107` (26 − 9 + 90). A coluna Code **não muda nada**: só a região de dados foi alterada. As instruções continuam buscando "o que estiver em `8($s0)`", seja lá o que for.
8	O programa para: `Runtime exception at 0x00400014: fetch address not aligned on word boundary 0x1001000a`. O endereço 0x1001000A não é múltiplo de 4 — fica no meio de duas palavras, e o MIPS não lê palavra desalinhada.
9	`Soma = 268501012`.
10	É o endereço de `soma`, `0x10010014`, escrito em decimal: 268 500 992 + 20 = 268 501 012. O `la` colocou em `$a0` o endereço, e o serviço 1 imprimiu o que estava em `$a0` — ele não tem como saber se é valor ou endereço.
11	Resposta pessoal. Exemplo: "`la` põe em `$a0` o lugar onde a soma está guardada; `lw` vai até esse lugar e põe em `$a0` a própria soma."

Sobre os códigos de máquina deste gabarito

Todos os programas e códigos foram conferidos no MARS 4.5. Se na sua máquina a coluna Basic mostrar `0x00000014` no lugar de 20, é só a exibição: desmarque *Settings → Values displayed in hexadecimal*.

Parte B — Desafio da semana

Uma solução possível (existem outras igualmente corretas):

```
        .data
nums:   .word  0, 0, 0          # espaço para os três números
msg:    .asciiz "Soma = "

        .text
        .globl main
main:
        la   $s0, nums          # $s0 = endereço do vetor

        li   $v0, 5             # serviço 5: ler inteiro
        syscall                 # o número lido chega em $v0
        sw   $v0, 0($s0)        # nums[0] = número lido

        li   $v0, 5
        syscall
        sw   $v0, 4($s0)        # nums[1]

        li   $v0, 5
        syscall
        sw   $v0, 8($s0)        # nums[2]

        lw   $t0, 0($s0)        # busca de volta na memória
        lw   $t1, 4($s0)
        add  $t2, $t0, $t1
        lw   $t1, 8($s0)
        add  $t2, $t2, $t1

        li   $v0, 4             # imprime o texto
        la   $a0, msg
        syscall

        li   $v0, 1             # imprime o número
        move $a0, $t2           # aqui o valor já está num registrador
        syscall

        li   $v0, 10
        syscall
```

Com as entradas 4, 10 e −3, o Run I/O mostra `Soma = 11`. Os números ficam em **`0x10010000`**, **`0x10010004`** e **`0x10010008`**. Na janela Data Segment, o −3 aparece como −3 com os valores em decimal, ou como `0xfffffffd` em hexadecimal — complemento de dois, encontro 2.

Dois pontos em que muita gente escorrega. O primeiro: o número lido chega em `$v0`, e a próxima chamada de sistema sobrescreve `$v0` com o número do serviço. Por isso ele precisa ser guardado **antes** do próximo `li $v0, …`. O segundo: `move $a0, $t2` é uma pseudoinstrução que copia um registrador para outro (vira `addu $4,$0,$10`). Quem escreveu `lw $a0, …` ou `add $a0, $t2, $zero` também está certo.

Guardar na memória e buscar de volta parece inútil com três números, que caberiam em registradores. Com mil números, não haveria registradores suficientes. O exercício treina o caminho que um programa de verdade precisa fazer.

Parte C — Exercícios de revisão

1. O endereço diz **onde** um dado está guardado na memória; o valor é **o próprio dado** guardado ali.

2. Porque cada palavra ocupa 4 bytes, e cada byte tem o seu próprio endereço. A palavra seguinte começa 4 bytes adiante.

3. `vetor[3]`: 0x10010000 + 4 · 3 = 0x10010000 + 12 = **`0x1001000C`**. `vetor[10]`: 0x10010000 + 4 · 10 = 0x10010000 + 40 = **`0x10010028`** (40 = 0x28).

4. `$t1` é o destino, que recebe a palavra lida. `12` é o deslocamento, em bytes. `$s0` é a base e é **ele** que contém um endereço. O endereço lido é `$s0 + 12`.

5. Falso. `sw` leva o valor de `$t2` para a memória, no endereço `$s0 + 20`. `$t2` não muda; quem muda é a memória.

6. Não, é uma pseudoinstrução. Ela vira `lui` + `ori`: o endereço tem 32 bits e o imediato de uma instrução tem só 16, então o montador carrega a metade de cima com `lui` e completa a metade de baixo com `ori`, usando `$at` como rascunho.

7. Porque 0x10010000 + 6 = 0x10010006 não é múltiplo de 4: a palavra começaria no meio de outra, e o MIPS exige alinhamento. Para os bytes 4 a 7, o deslocamento é **4**: `lw $t0, 4($s0)`.

8. Em binário, `1010 1110 0000 1010 0000 0000 0001 0100`. Reagrupando em 6-5-5-16:

op	rs	rt	imediato
101011	**10000**	**01010**	**0000000000010100**
43 → sw	16 → $s0	10 → $t2	20

> A instrução é `sw $t2, 20($s0)`. Na ordem do texto, primeiro rt, depois o deslocamento e a base entre parênteses. É a instrução que grava a soma no programa da aula.

9. (a) `$v0` = 1; `$a0` = o valor de `$t5` (por exemplo, `move $a0, $t5`). (b) `$v0` = 4; `$a0` = o endereço do texto (`la $a0, aviso`). (c) `$v0` = 5; não precisa de `$a0`, e o número digitado chega em `$v0`. (d) `$v0` = 10; não precisa de `$a0`.

10. `$t1` = 20 (x[1]), `$t2` = 30 (x[2]), `$t3` = 50. O `sw` grava 50 na primeira palavra de `x`, que fica **50, 20, 30**. Só a primeira palavra mudou.

11. Uma solução:

```
        .data
vetor:  .word  7, 3, 9, 2, 5
        .text
main:   la   $s0, vetor
        lw   $t0, 0($s0)        # $t0 = vetor[0] = 7
        lw   $t1, 16($s0)       # $t1 = vetor[4] = 5
        sw   $t1, 0($s0)        # vetor[0] = 5
        sw   $t0, 16($s0)       # vetor[4] = 7
        li   $v0, 10
        syscall
```

> São necessários dois temporários. Com um só, a primeira escrita apagaria um dos valores antes de ele ser guardado em outro lugar: ao gravar o 5 em `vetor[0]`, o 7 se perderia. É o mesmo motivo pelo qual, em C, uma troca precisa de uma variável auxiliar.